MATH50009.01 数值算法与案例分析 II Spring 2026

Lecture 1 数值代数基本问题与非线性方程解法 — 2026.03.03

教授：邵美悦

Scribe: 路人甲

目录

1 基础知识回顾 **1**

1.1 数值代数基本问题 1

1.2 误差分析 2

2 案例分析：如何高效地计算数学常数？ **2**

3 非线性方程的求解 **5**

3.1 二分法 (bisection) 5

3.2 试位法 (Regula Falsi / False Position) 6

3.3 不动点迭代法 (Fixed-Point Iteration) 6

3.4 Newton 法 6

1 基础知识回顾

1.1 数值代数基本问题

数值代数的基本问题可以概括为以下几个方面：

1. **线性代数问题**：求解线性方程组 $Ax = b$，最小二乘问题 $\min_x \|b - Ax\|_2$，特征值问题 $Ax = x\lambda$，奇异值分解 $Av = u\sigma$ 与 $A^*u = v\sigma$，矩阵函数与矩阵方程求解等。
2. **微积分与非线性问题**：求解非线性方程 $f(x) = 0$，数值微分与数值积分，数值微分方程，以及高精度数值计算等。

在上个学期，我们主要关注了线性代数问题。本学期，我们将重点关注微积分与非线性问题。本讲主要介绍非线性方程的求解方法。在介绍非线性方程解法之前，我们先回顾一些基础知识。

1.2 误差分析

在上个学期，我们已经介绍了误差分析的基本概念，包括绝对误差、相对误差、向前向后误差、条件数等，不过当时我们主要关注了线性问题的误差分析。本学期，我们将继续使用这些概念来分析非线性问题的误差。下面我们不作过多介绍，直接给出定义。

1.1 定义 绝对误差、相对误差

设 $\hat{x}$ 是 x 的近似值，则绝对误差定义为 $\varepsilon = \|\hat{x} - x\|$，相对误差定义为 $\delta = \frac{\varepsilon}{\|x\|}$（当 $x \neq 0$ 时）。

1.2 定义 向前误差、向后误差

设 $\hat{y}$ 是 $y = f(x)$ 的近似值，则向前误差定义为 $\varepsilon_{\text{forward}} = \|\hat{y} - y\|$（绝对误差）或 $\varepsilon_{\text{forward}} = \frac{\|\hat{y}-y\|}{\|y\|}$（相对误差）。
若存在一个输入 $\hat{x}$ 使得 $\hat{y} = f(\hat{x})$ 或 $\hat{y} = f(\hat{x}) + \Delta y$，则向后误差定义为 $\varepsilon_{\text{backward}} = \|\hat{x} - x\|$（绝对误差）或 $\varepsilon_{\text{backward}} = \frac{\|\hat{x}-x\|}{\|x\|}$（相对误差）。

2 案例分析：如何高效地计算数学常数？

2.1 例

试计算 $\sqrt{2}$ 的近似值。

解. 最直观的方法是利用 *Taylor* 展开。已知

$$\sqrt{1+x} = 1 + \frac{1}{2}x - \frac{1}{8}x^2 + \frac{f'''(\xi)}{3!}x^3$$

直接代入 $x = 1$ 并不好，因为这个级数的收敛半径是 1，往往需要很多项才能得到较好的近似。我们可以通过一些变换来加速收敛，如

$$\sqrt{2} = 1.4 \cdot \sqrt{1 + \frac{0.04}{1.96}}$$

然后对 $\sqrt{1 + \frac{0.04}{1.96}}$ 进行 *Taylor* 展开，这样就能更快地得到 $\sqrt{2}$ 的近似值。

然而，*Taylor* 展开虽然可行，但并不是最高效的方法。每迭代一步，我们只增加了一个阶数的精度。古巴比伦人提出了一个更高效的方法，被称为巴比伦迭代法 *(Babylonian method)*，之后我们会看到它是 *Newton* 法的一个特例。它的迭代格式是

$$a_{n+1} = \frac{1}{2}\left(a_n + \frac{A}{a_n}\right)$$

其中 A 是我们要开方的数。下面我们将分析这个方法的收敛性。显然我们可以猜测：当 $a_0 > 0$ 时，a_n 会收敛到 $\sqrt{A}$；当 $a_0 < 0$ 时，a_n 会收敛到 $-\sqrt{A}$. 因此，不妨考虑

$$a_{n+1} - \sqrt{A} = \frac{1}{2}\left((a_n - \sqrt{A}) + \left(\frac{A}{a_n} - \sqrt{A}\right)\right) = \frac{1}{2a_n}(a_n - \sqrt{A})^2 \tag{1}$$

同理，我们也可以得到

$$a_{n+1} + \sqrt{A} = \frac{1}{2a_n}(a_n + \sqrt{A})^2 \tag{2}$$

现在我们对 (1) 进行分析。注意到当 $a_n > 0$ 时，由均值不等式，$a_n \geq \sqrt{A}$，且

$$\frac{a_n - \sqrt{A}}{2a_n} \in \left[0, \frac{1}{2}\right)$$

因此

$$a_{n+1} - \sqrt{A} < \frac{1}{2}(a_n - \sqrt{A})$$

因此数列 $\{a_n\}$ 是全局线性收敛的；同样地，当 $a_0 < 0$ 时，a_n 也是全局线性收敛的。

但在实际操作中，巴比伦迭代法的收敛速度似乎比线性收敛要快得多，有时甚至达到了二次收敛的速度。我们将 (1) 和 (2) 两式对除，有

$$\frac{a_{n+1} - \sqrt{A}}{a_{n+1} + \sqrt{A}} = \left(\frac{a_n - \sqrt{A}}{a_n + \sqrt{A}}\right)^2$$

因此数列 $\left\{\frac{a_n - \sqrt{A}}{a_n + \sqrt{A}}\right\}$ 是平方收敛的，当 $a_0 > 0$ 时，a_n 将以平方收敛的速度收敛到 $\sqrt{A}$；当 $a_0 < 0$ 时，a_n 将以平方收敛的速度收敛到 $-\sqrt{A}$。因此，巴比伦迭代法是一种非常高效的计算 $\sqrt{A}$ 的方法。

另外，若 $A < 0$，若 $a_0 \in \mathbb{R}$，则 a_n 将会发散；若 $a_0 \in \mathbb{C} \setminus \mathbb{R}$，则 a_n 将会根据 a_0 的位置收敛到 $\sqrt{-A}\mathrm{i}$ 或 $-\sqrt{-A}\mathrm{i}$.

注：

1. 所谓的“线性收敛”是指误差在每次迭代中以一个常数因子 r 减少，反映在有效数字上，就是每次迭代大约在某个进制下增加一个有效数字。线性收敛的迭代格式通常满足 $\|x_{n+1} - x_*\| \leq r\|x_n - x_*\|$ 其中 $0 < r < 1$.

2. 而“平方收敛”是指误差在每次迭代中以误差的平方减少，反映在有效数字上，就是每次迭代大约在某个进制下增加两倍的有效数字。平方收敛的迭代格式通常满足 $\|x_{n+1} - x_*\| \leq C\|x_n - x_*\|^2$ 其中 $C > 0$.

3. 巴比伦迭代法具有全局线性收敛性和局部二次收敛性。例如，当 $A = 2$，初值 $a_0 = 1$ 时，序列的前 8 项是

   1
   1.5
   1.41666666666666666666666666666666666666666666666666666666666675
   1.414215686274509803921568627450980392156862745098039215686274 5
   1.41421356237468991062629557889013491011655962211574404458490 57
   1.41421356237309504880168962350253024361498192577619742849828 90
   1.4142135623730950488016887242096980785696718753772340015610125
   1.414213562373095048801688724209698078569671875376948073176679 6

   其中加粗部分是有效数字。显然有效数字大约是成倍增长的。

2.2 例

计算 $\ln 2$ 的近似值。

解. 考虑 *Taylor* 展开

$$\ln(1+x) = x - \frac{1}{2}x^2 + \frac{1}{3}x^3 - \cdots \tag{3}$$

直接代入 $x = 1$ 并不好，这个级数的收敛半径是 1，因此需要计算很多项才能得到较好的近似。考虑

$$\ln(1-x) = -x - \frac{1}{2}x^2 - \frac{1}{3}x^3 - \cdots \tag{4}$$

我们将 (3) 和 (4) 两式相减，得到

$$\ln\frac{1+x}{1-x} = 2\left(x + \frac{1}{3}x^3 + \frac{1}{5}x^5 + \cdots\right)$$

代入 $x = \frac{1}{3}$，我们就得到了 $\ln 2$ 的一个更高效的近似方法。

2.3 例

考虑 Basel 问题

$$\sum_{n=1}^{\infty}\frac{1}{n^2} = \frac{\pi^2}{6}.$$

通过此式计算 π 的近似值。

解. 考虑 $a_n = \sum_{k=1}^{n} \frac{1}{k^2}$，记 $r_n = \sum_{k=n+1}^{+\infty}$，则

$$
\begin{aligned}
r_n &= \frac{1}{(n+1)^2} + \frac{1}{(n+2)^2} + \cdots \\
&< \frac{1}{n(n+1)} + \frac{1}{(n+1)(n+2)} + \cdots \\
&= \frac{1}{n} \\
&> \frac{1}{(n+1)(n+2)} + \frac{1}{(n+2)(n+3)} + \cdots \\
&= \frac{1}{n+1}
\end{aligned}
$$

因此 $\frac{1}{n+1} < r_n < \frac{1}{n}$，我们需要计算一百项才能得到两位有效数字。

但是，既然我们知道 r_n 的渐进行为，我们就可以通过一些变换来加速收敛。考虑

$$
\tilde{a}_n = a_n + \frac{1}{n+1}
$$

于是 $\tilde{r}_n = r_n - \frac{1}{n+1} \in \left(0, \frac{1}{n(n+1)}\right)$，现在我们只需十次迭代就能得到两位有效数字了。

当然，我们也可以考虑把 r_n 展开到更高阶，然后在 a_n 中加入更高阶的项来加速收敛。

注：这种通过分析误差的渐近行为来加速收敛的方法非常重要。在上面的例子中，我们主要利用裂项来分析；但在更一般的场景下，我们可能需要利用 Taylor 展开或积分来分析误差的渐近行为。例如，$\zeta(\mathrm{e}) = \sum_{k=1}^{+\infty} 1/k^{\mathrm{e}}$ 的误差可以由积分来估计：

$$
\begin{aligned}
r_n &= \sum_{k=n+1}^{+\infty} \frac{1}{k^{\mathrm{e}}} \\
&< \int_n^{+\infty} \frac{1}{x^{\mathrm{e}}} \mathrm{d}x = \frac{1}{(\mathrm{e}-1)n^{\mathrm{e}-1}} \\
&> \int_{n+1}^{+\infty} \frac{1}{x^{\mathrm{e}}} \mathrm{d}x = \frac{1}{(\mathrm{e}-1)(n+1)^{\mathrm{e}-1}}
\end{aligned}
$$

3 非线性方程的求解

3.1 二分法 (bisection)

该算法的基本思想是逐步缩小包含根的区间。设 $f : [a, b] \to \mathbb{R}$ 是连续函数，且 $f(a)f(b) < 0$. 我们不断取区间中点 $c = (a+b)/2$，然后取异号区间继续迭代，直到区间长度小于容差。该方法显然具有全局线性收敛性：每次迭代都会让区间长度变为上次的一半，在二进制上增加一位有效数字。这种方法极其稳定，但收敛速度较慢。

此外，倘若函数求值存在误差，即计算 $f(c)$ 时存在误差，算出来的是 $\hat{f}(c)$，当 $f(c) \approx 0$ 时，有可能会把 $f(c)$ 的符号算错，从而导致算法失败。这个问题没有好的解决办法。一个不是办法的办法是，当 $|\hat{f}(c)| < \varepsilon$ 时停止迭代，但这并不能保证算法的正确性：当所在区域斜率比较平缓时，c 可能和真实零点差距比较大，算法也只能无奈妥协。

3.2 试位法 (Regula Falsi / False Position)

和二分法类似，唯一的区别是用割线与 x 轴的交点来替代重中点。注意：它不一定比二分法快!

3.3 不动点迭代法 (Fixed-Point Iteration)

以上两种依赖区间端点符号变号的方法，都无法用来寻找复数根。而不动点迭代法不依赖于函数值的符号，因此可以用来寻找复数根。假设我们要求解的方程是 $F(x) = 0$，我们将其等价地改写成不动点格式 $x = G(x)$. 于是迭代格式自然地定义为

$$x_{k+1} = G(x_k).$$

这个算法在什么时候能够收敛呢？我们利用 Lagrange 中值定理展开相邻两项的误差：

$$x_{k+1} - x_k = G(x_k) - G(x_{k-1}) = G'(\xi)(x_k - x_{k-1}), \quad \xi \in (x_{k-1}, x_k).$$

直觉上，我们希望 $|G'(\xi)| < 1$，这样会使误差缩小。但仅仅保证每一项 $|G'(\xi_i)| < 1$ 是不够的（因为它们的乘积未必趋于 0）。我们需要一个更严格的收敛条件：必须存在一个严格小于 1 的一致上界 $\gamma < 1$. 若 $\max_{x \in R(G)} |G'(x)| < 1$，则

$$x_{k+1} - x_* = G'(\xi)(x_k - x_*) \leq \gamma |x_k - x_*|$$

于是

$$|x_k - x_*| \leq \gamma^k |x_0 - x_*|$$

因此数列 $\{x_k\}$ 是全局线性收敛的。

3.4 Newton 法

Newton 法的核心是用切线来逼近函数。我们在 x_k 处对函数 $F(x)$ 进行线性近似，得到切线方程：

$$y - F(x_k) = F'(x_k)(x - x_k).$$

将切线与 x 轴的交点作为下一次迭代的点，即

$$x_{k+1} = x_k - \frac{F(x_k)}{F'(x_k)}.$$

从不动点的视角来看，Newton 法相当于取特定的迭代函数

$$G(x) = x - \frac{F(x)}{F'(x)}.$$

根据前面的分析，我们计算其导数

$$G' = \frac{FF''}{(F')^2}.$$

当 x 逼近真实的单根 x_* 时，$F(x_*) = 0$，从而 $G'(x_*) = 0$。显然在根的局部邻域内满足 $|G'(x)| < 1$，因此具有局部线性收敛性。更进一步地，我们还可以证明 Newton 法具有局部二次收敛性。

我们考虑对下式进行 Taylor 展开：

$$\begin{aligned} x_{k+1} - x_* &= G(x_k) - G(x_*) \\ &= G'(x_*)(x_k - x_*) + \frac{G''(\xi)}{2}(x_k - x_*)^2 \end{aligned}$$

因为 $G'(x_*) = 0$，所以第一项为零，因此

$$|x_{k+1} - x_*| \leq \frac{\max_{x \in R(G)} |G''(x)|}{2} |x_k - x_*|^2$$

对于一般的不动点迭代函数，我们还可以做如下分类：

1. 若 $G'(x_*) = 0$，则迭代具有局部二次收敛性。
2. 若 $0 < |G'(x_*)| < 1$，则迭代具有局部线性收敛性。
3. 若 $|G'(x_*)| \geq 1$，则数列不收敛。

Newton 法具有极快的局部二次收敛性（针对单根），但其全局收敛性取决于目标函数的具体性质。回顾第 3 节，计算 $\sqrt{2}$ 的巴比伦迭代法，本质上就是对函数 $F(x) = x^2 - A$ 应用 Newton 法的结果。

有关 Newton 法的全局收敛性，感兴趣的读者可以搜索 *Newton Fractal* 来了解更多相关内容。